BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 03b — Tubes et concurrence entre processus

```
bash — R3.05
$ ls /etc | wc -l
214
$ mkfifo canal
$ ls -l canal
prw-r--r-- 1 cruz info 0 canal
```

Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

Au programme

01

Les tubes

Principe, pipe, lecture et écriture, interblocages.

02

Les redirections

dup, dup2, et la mise en œuvre de ls | more.

03

Les tubes nommés et les IPC

mkfifo, et un panorama des autres mécanismes de communication.

04

La concurrence

Conditions de concurrence, sections critiques, exclusion mutuelle.

Prérequis : CM 02 (descripteurs, read, write, close) et CM 03a (fork, exec, wait).

01

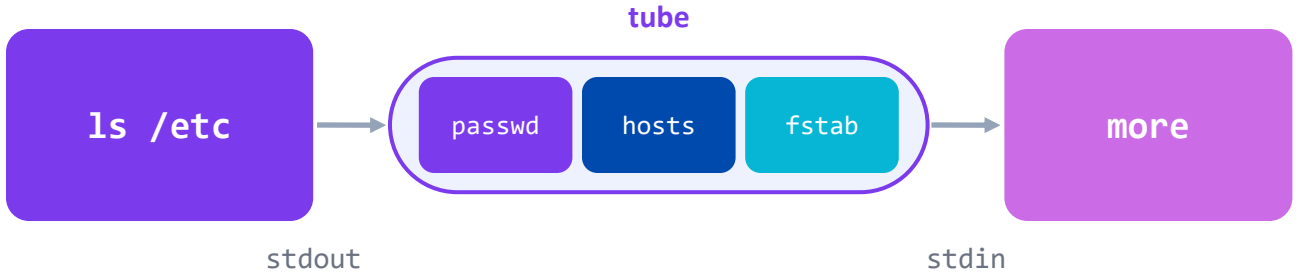
Les tubes

Faire communiquer deux processus

Les tubes, vous les utilisez déjà

```
$ ls /etc | more
```

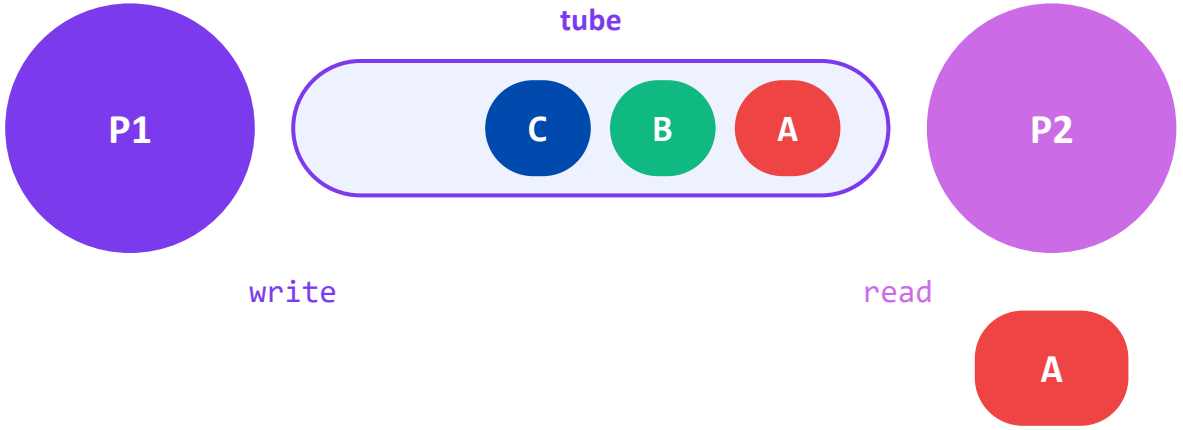
- Le shell lance ls et more en même temps
- La sortie standard de ls est reliée à l'entrée standard de more par un tube (pipe)
- ls écrit sans savoir qu'il écrit dans un tube ; more lit sans savoir qu'il lit un tube



Si more est plus lent, le tube se remplit et ls est mis en attente : le tube synchronise les deux processus.

Le principe : un canal FIFO

- Un tube est un tampon dans le noyau, avec deux extrémités : une pour écrire, une pour lire
- Il est décrit par un i-nœud, mais il n'a pas de nom dans l'arborescence
- Les données forment un flot d'octets sans structure
- FIFO : la première donnée écrite est la première lue
- Capacité finie (64 Kio sous Linux) : un tube peut être plein



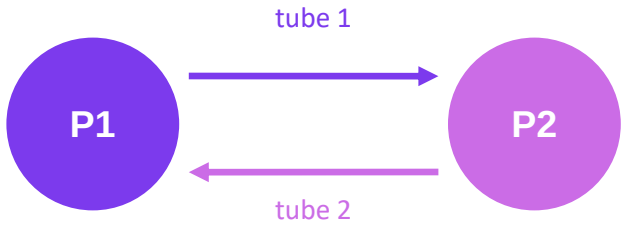
P2 lit d'abord A, le premier écrit

Lecture effaçante : lire dans un tube retire les données. On ne peut pas les relire ni se déplacer (lseek interdit).

Sens unique, lecteurs et rédacteurs

Unidirectionnel

Un tube va dans un seul sens. Pour dialoguer, il faut deux tubes.



Plus aucun lecteur

Les lecteurs sont les descripteurs ouverts en lecture sur le tube. S'il n'y en a plus, écrire est impossible.

`write → signal SIGPIPE`
qui tue le processus : « Broken pipe ». Si SIGPIPE est ignoré, write renvoie -1 (EPIPE).

Plus aucun rédacteur

Les rédacteurs sont les descripteurs ouverts en écriture. S'il n'y en a plus, le tube est « fermé » côté écriture.

`read → renvoie 0`
une fois le tube vidé : c'est la fin de fichier, comme pour un fichier ordinaire.

Conséquence : un descripteur oublié ouvert compte comme un lecteur ou un rédacteur. D'où la règle : fermer les descripteurs inutiles.

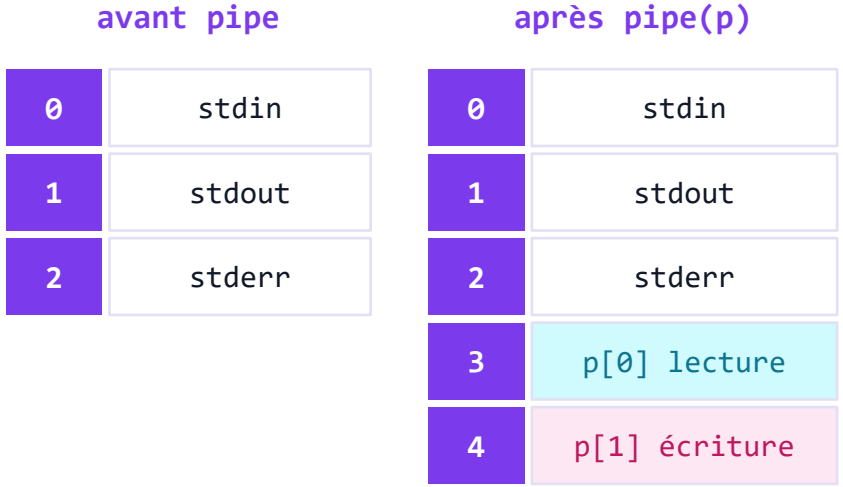
Créer un tube : pipe

```c
#include <unistd.h>

int pipe(int p[2]);     // 0 si succès, -1 si erreur
```

- Le paramètre est un tableau de deux entiers, rempli par pipe
- p[0] : le descripteur pour lire dans le tube
- p[1] : le descripteur pour écrire dans le tube
- Moyen mnémotechnique : 0 comme stdin (lecture), 1 comme stdout (écriture)

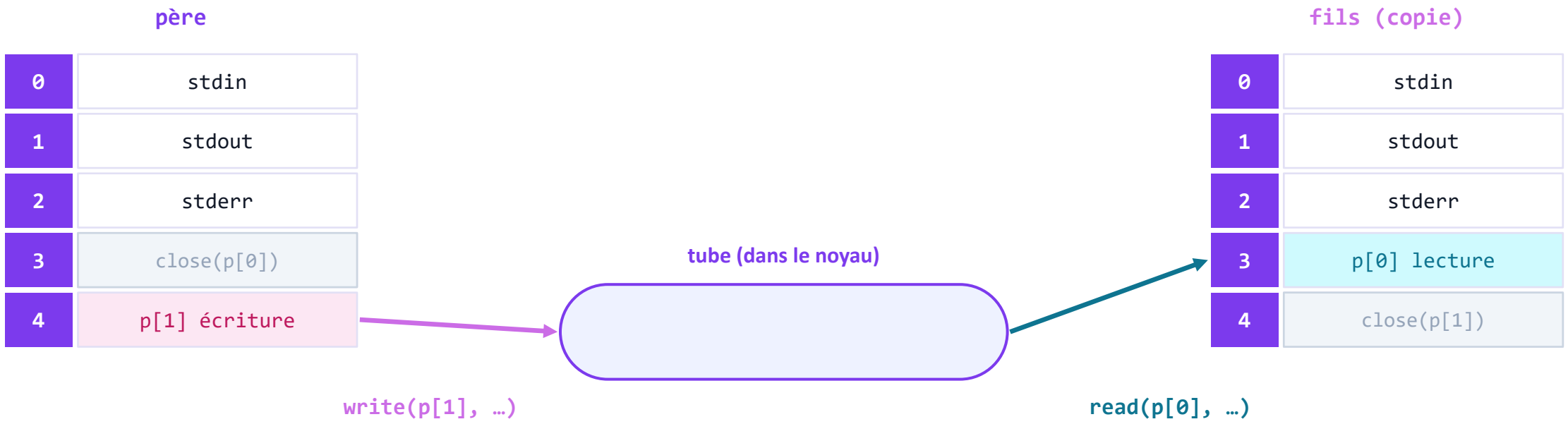
```c
int p[2];
if (pipe(p) == -1) { perror("pipe"); exit(EXIT_FAILURE); }
```



Pas de nom, donc pas d'open : un processus obtient un descripteur de tube soit en le créant, soit par héritage (fork). Seuls le créateur et ses descendants peuvent l'utiliser.

pipe + fork : le schéma classique

1. pipe(p)
2. fork()
3. fermer le côté inutile
4. communiquer



Après le fork, le tube a 2 lecteurs et 2 rédacteurs. Il faut en fermer pour obtenir un canal propre : un seul rédacteur (le père), un seul lecteur (le fils).

Exemple : le père envoie un mot au fils

tube.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
    int p[2];
    if (pipe(p) == -1) { perror("pipe"); exit(1); }
    if (fork() == 0) {                    // fils : lecteur
        close(p[1]);
        char mot[32];
        ssize_t n = read(p[0], mot, sizeof mot - 1);
        mot[n > 0 ? n : 0] = '\0';
        printf("fils : mot reçu [%s]\n", mot);
        close(p[0]);
        exit(0);
    }
    close(p[0]);                          // père : écrivain
    write(p[1], "Bonjour", 7);
    close(p[1]);
    wait(NULL);
    return 0;
}
```

```
$ ./tube
fils : mot reçu [Bonjour]
```

Le fils lit

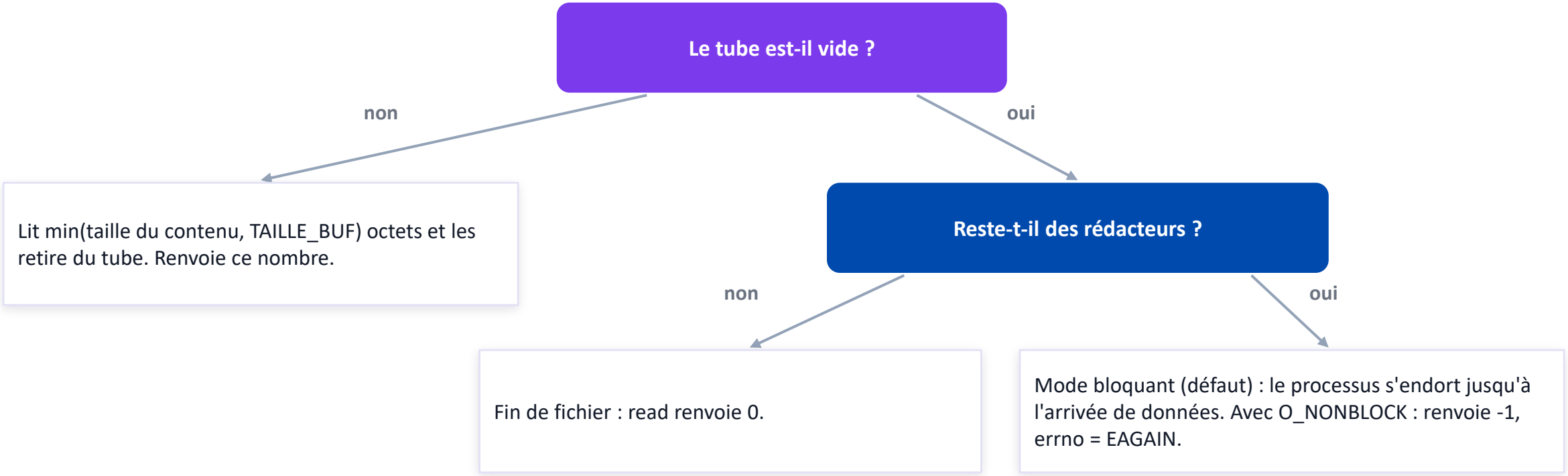
Il ferme p[1], lit au plus 31 octets, ajoute '\0' : read ne termine pas la chaîne.

Le père écrit

Il ferme p[0], écrit 7 octets, ferme p[1] (le fils verra la fin), puis attend son fils.

Lire dans un tube : les règles

```
nb_lu = read(p[0], buf, TAILLE_BUF);
```



Le cas « non, aucun rédacteur » est la raison pour laquelle il faut fermer p[1] dans le lecteur.

Écrire dans un tube : les règles

```
nb_ecrit = write(p[1], buf, n);
```

01

Aucun lecteur

Le signal SIGPIPE est envoyé à l'écrivain : par défaut, il le termine (« Broken pipe »). S'il est ignoré, write renvoie -1 avec errno = EPIPE.

02

Mode bloquant (défaut)

Si le tube est plein, l'écrivain s'endort jusqu'à ce qu'il se vide. Si n ≤ PIPE_BUF (4 096 octets sous Linux), l'écriture est atomique : jamais mélangée avec celle d'un autre processus.

03

Mode non bloquant

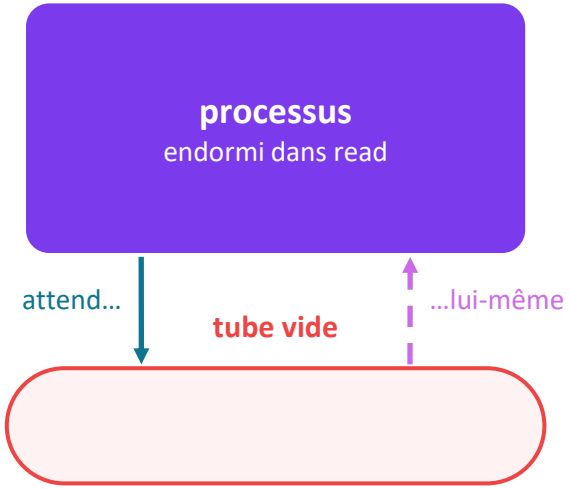
Avec O_NONBLOCK : si la place manque, write renvoie -1 (EAGAIN) quand n ≤ PIPE_BUF, ou écrit ce qu'il peut (écriture partielle) quand n > PIPE_BUF.

Bonne pratique : comme pour un fichier, testez la valeur renvoyée par write, et gérez le cas où elle est inférieure à n.

Attention à l'interblocage (1)

blocage1.c

```c
int p[2];
pipe(p);
/* le tube est vide ; le seul écrivain est ce processus,
   qui n'a pas fermé p[1] */
read(p[0], &ch, 1);      // BLOCAGE pour toujours
write(p[1], &ch, 1);     // jamais atteint
```

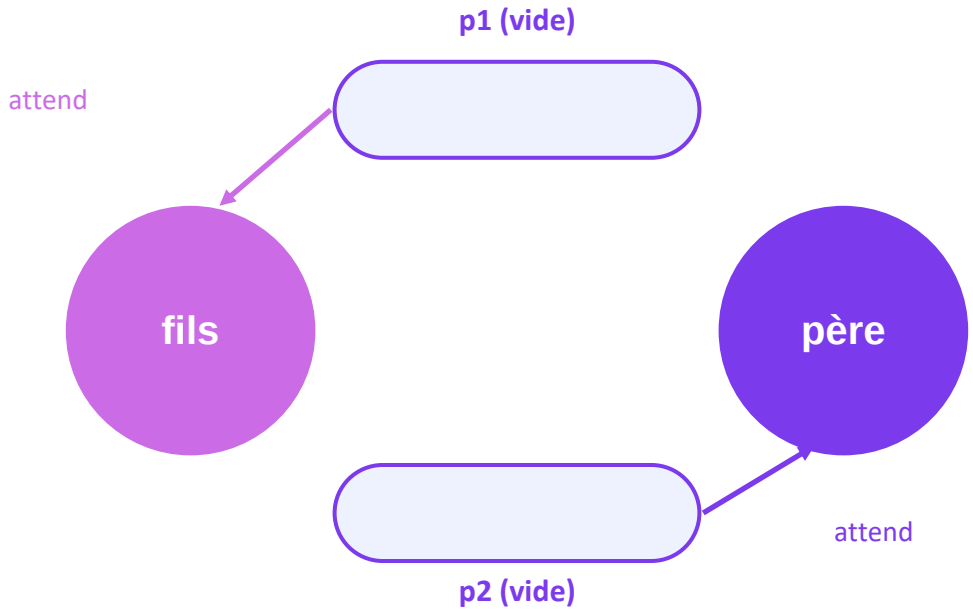


Pourquoi ?
read ne renvoie 0 que s'il n'y a plus aucun rédacteur. Or ce processus est lui-même rédacteur (p[1] est ouvert) : read attend donc indéfiniment des données que personne n'écrira. C'est un interblocage (deadlock), et il ne produit aucun message d'erreur.

Attention à l'interblocage (2)

blocage2.c

```c
pipe(p1);
pipe(p2);
if (fork() == 0) {
    read(p1[0], &ch, 1);    // BLOCAGE : p1 est vide
    write(p2[1], &ch, 1);
} else {
    read(p2[0], &ch, 1);    // BLOCAGE : p2 est vide
    write(p1[1], &ch, 1);
}
```



Solution : définir un ordre (l'un écrit d'abord, l'autre lit d'abord), et fermer dans chaque processus les extrémités qu'il n'utilise pas.

Cycle d'attente
Chacun attend une donnée que l'autre n'enverra qu'après avoir reçu la sienne.

La règle d'or des tubes

Fermez toujours les descripteurs inutiles !

01

Sinon le lecteur ne voit jamais la fin

Un p[1] resté ouvert, même dans le lecteur lui-même, empêche read de renvoyer 0 : le programme se bloque.

02

Sinon l'écrivain n'est jamais prévenu

Un p[0] resté ouvert ailleurs empêche SIGPIPE : l'écrivain remplit le tube et s'endort.

03

Sinon la table déborde

Le nombre de descripteurs par processus est limité, et chaque fork les duplique.

02

Les redirections

dup, dup2 : brancher un tube sur stdin ou stdout

Copier un descripteur : dup et dup2

```c
#include <unistd.h>

int dup(int ancien);
int dup2(int ancien, int nouveau);
```

dup(ancien)

Crée une copie dans le plus petit numéro libre et renvoie ce numéro (-1 si erreur).

dup2(ancien, nouveau)

Ferme nouveau s'il est ouvert, puis en fait une copie de ancien, en une seule opération. À préférer.

- L'ancien et le nouveau descripteur sont interchangeables
- Ils partagent le même fichier ouvert, donc la même position courante
- Les programmes écrivent sur le descripteur 1 sans savoir ce qu'il y a derrière : c'est ce qui permet les redirections

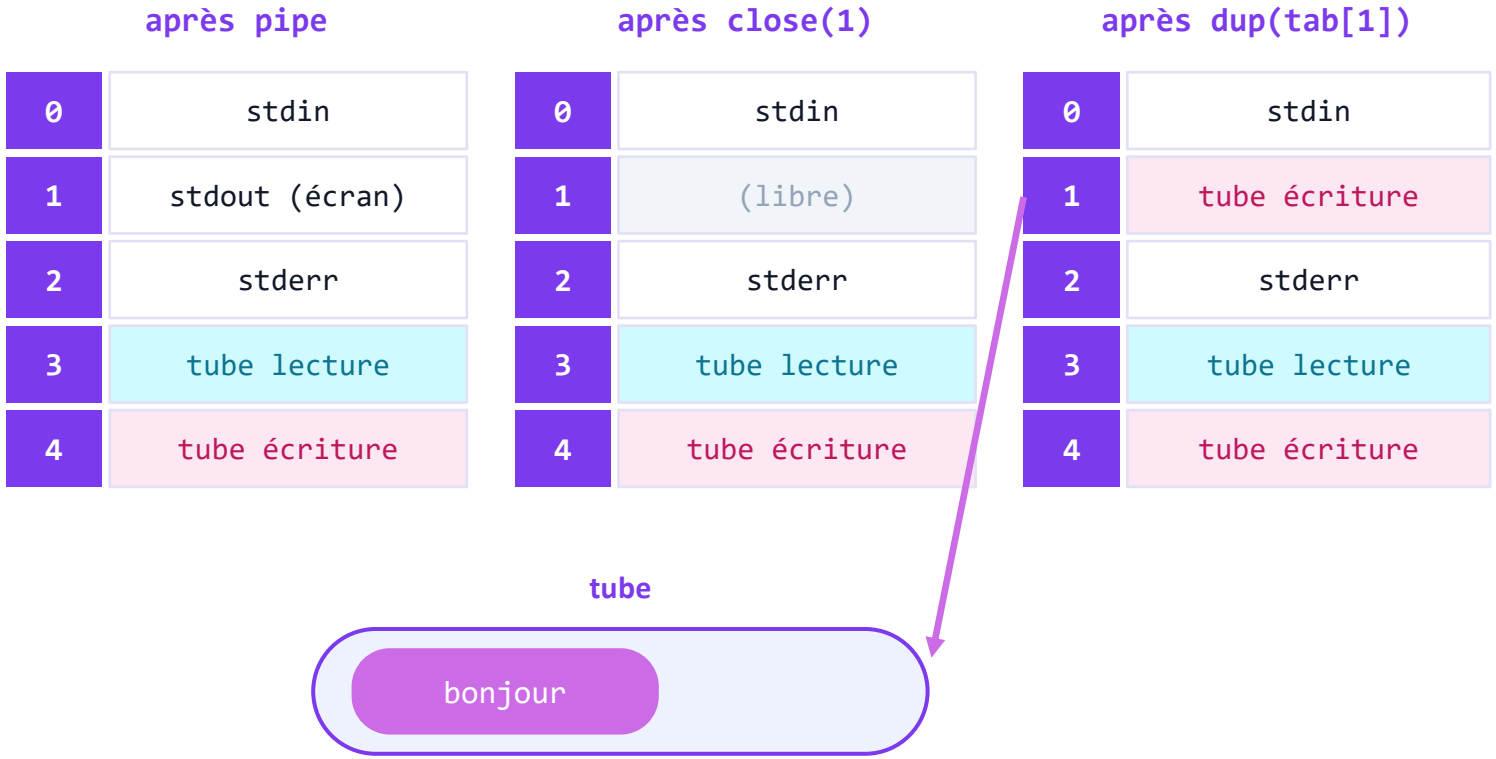
comme ./prog > log.txt

```c
int fd = open("log.txt", O_WRONLY | O_CREAT, 0644);
dup2(fd, 1);          // stdout -> log.txt
close(fd);
printf("dans le fichier\n");
```

Rediriger stdout vers un tube, pas à pas

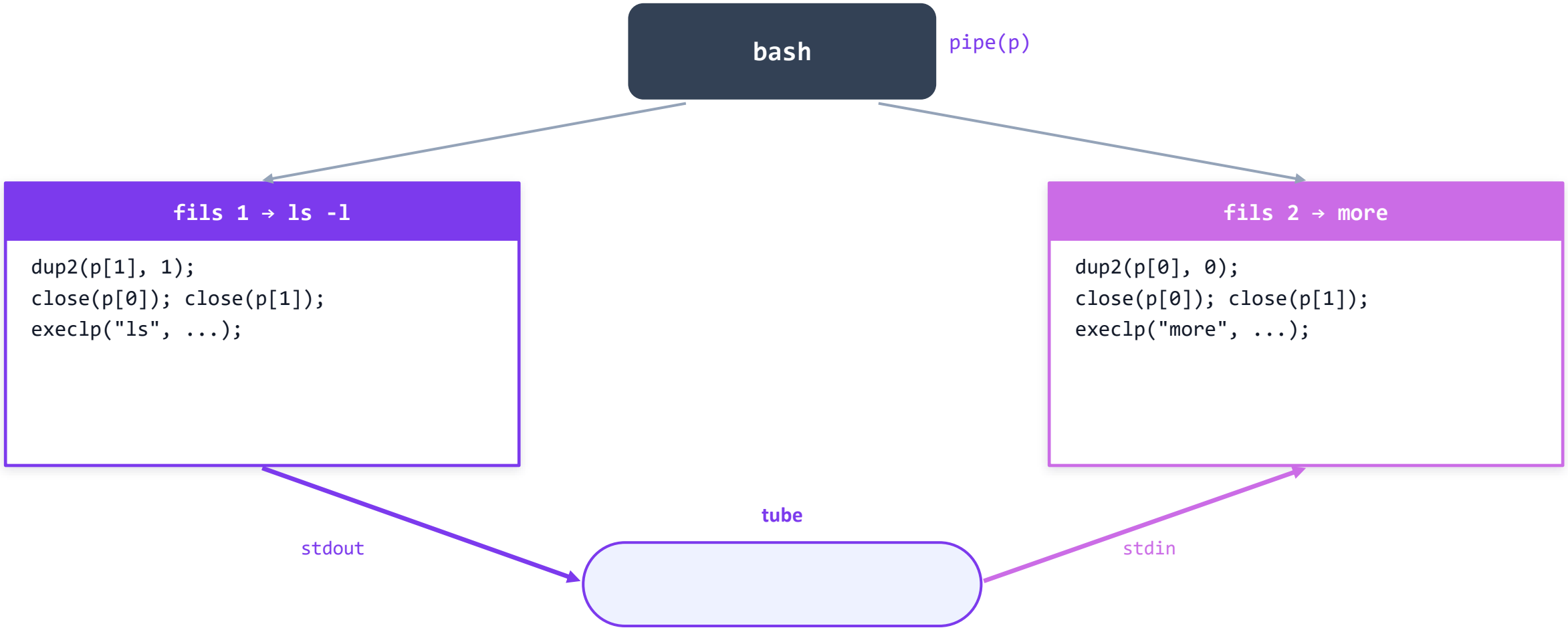
```c
pipe(tab);
close(1);
dup(tab[1]);
printf("bonjour");
```

printf écrit sur le descripteur 1… qui n'est plus l'écran mais le tube : « bonjour » n'apparaît pas à l'écran.




```c
pipe(tab);
dup2(tab[1], STDOUT_FILENO);   // les 2 étapes en une
close(tab[1]);
```

Comment le shell exécute ls -l | more



bash : close(p[0]); close(p[1]); puis wait × 2

ls -l | more en C

pipeline.c

```c
int p[2];
pipe(p);
if (fork() == 0) {                  // fils 1 : ls -l
    dup2(p[1], STDOUT_FILENO);
    close(p[0]); close(p[1]);
    execlp("ls", "ls", "-l", NULL);
    perror("execlp ls"); exit(1);
}
if (fork() == 0) {                  // fils 2 : more
    dup2(p[0], STDIN_FILENO);
    close(p[0]); close(p[1]);
    execlp("more", "more", NULL);
    perror("execlp more"); exit(1);
}
close(p[0]); close(p[1]);           // le père n'utilise pas le tube
wait(NULL); wait(NULL);
```

Les descripteurs survivent à exec

Après dup2, ls écrit sur le descripteur 1 sans rien savoir du tube. C'est tout l'intérêt.

Si le père oublie la ligne surlignée… il reste un rédacteur sur le tube : more ne voit jamais la fin de fichier et attend indéfiniment, et le père aussi dans son wait.

03

Tubes nommés et IPC

Faire communiquer des processus sans lien de parenté

Les tubes nommés : mkfifo

```c
#include <sys/stat.h>

int mkfifo(const char *chemin, mode_t mode);
```

- Un tube nommé (FIFO) a un nom dans l'arborescence (type p dans ls -l)
- N'importe quel processus qui a les droits peut l'ouvrir avec open, même sans lien de parenté
- open en lecture attend qu'un écrivain ouvre aussi le tube, et inversement
- Ensuite, il fonctionne exactement comme un tube

terminal 1

```
$ mkfifo canal
$ ls -l canal
prw-r--r-- 1 cruz info 0 canal
$ cat < canal
```

… attend un écrivain

terminal 2

```
$ echo "coucou" > canal
```

terminal 1

```
coucou
$
```

Le fichier canal ne contient rien : les données transitent par le noyau.

Panorama des IPC

IPC : Inter-Process Communication. Les mécanismes fournis par le noyau pour faire communiquer des processus.

Tubes
ce CM

pipe, mkfifo : un flot d'octets à sens unique.

Signaux
CM 03a

kill, sigaction : une notification, sans données.

Mémoire partagée
POSIX

shm_open, mmap : une zone de mémoire commune. Le plus rapide, mais il faut synchroniser.

Sémaphores
POSIX

sem_open, sem_wait, sem_post : des compteurs pour synchroniser.

Files de messages
POSIX

mq_open, mq_send : des messages typés, avec priorités.

Sockets
séance 9

socket, connect : communication locale ou par le réseau.

IPC System V (shmget, semget, msgget) : l'ancienne API de ces trois mécanismes, encore très répandue. La commande ipcs les liste.

04

La concurrence

Quand plusieurs processus accèdent à la même donnée

Les conditions de concurrence

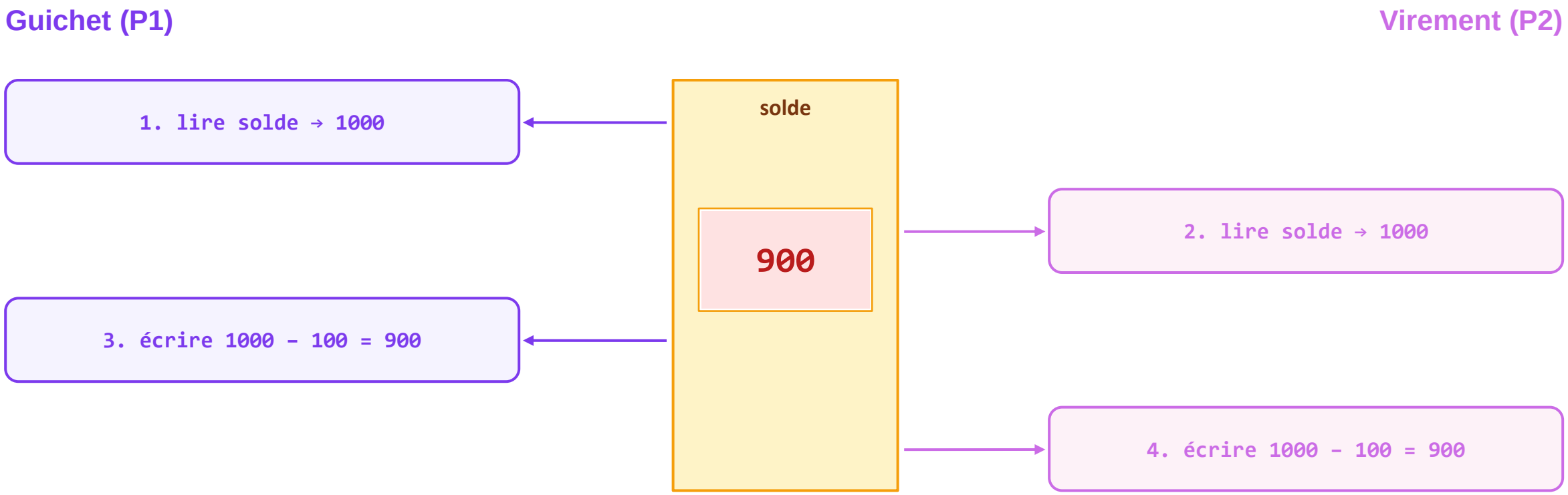
- Des processus peuvent partager un espace de stockage commun : une variable en mémoire partagée, un fichier…
- Le problème est le même quel que soit l'emplacement de la donnée
- Une condition de concurrence (race condition) : le résultat dépend de l'ordre exact dans lequel les processus s'exécutent



Pourquoi c'est sournois : le bug n'apparaît que pour certains entrelacements. Le programme marche 999 fois sur 1 000.

On parle ici de processus ; tout s'applique aussi aux threads (séance 8).

Exemple 1 : le compte bancaire



Solde final : 900 € au lieu de 800 €. Le retrait de P1 a été perdu : P2 a écrasé son résultat.

Exemple 2 : le spouleur d'impression

- Pour imprimer, un processus range le nom de son fichier dans le répertoire de spoule, à la case in (prochaine case libre), puis fait in = in + 1
- Le démon d'impression imprime la case out

0	toto.txt
1	file.c
2	test.pdf
3	P1.c
4	

```
out = 0
in  = 3
```

4

1. P1 lit in → 3, puis il est interrompu

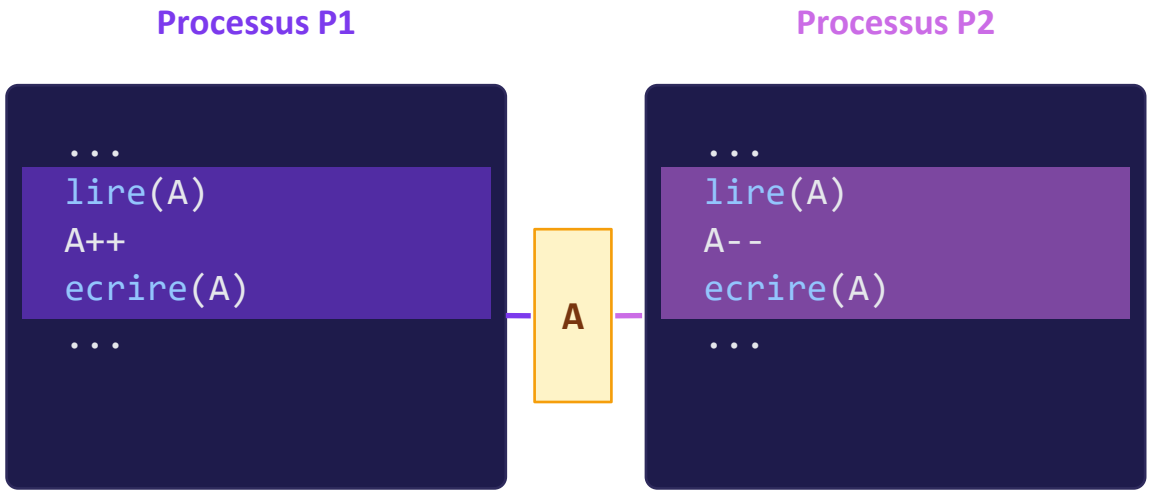
2. P2 lit in → 3, range P2.c en case 3, met in à 4

3. P1 reprend : range P1.c en case 3 (sa valeur lue), met in à 4

P2.c ne sera jamais imprimé, et aucun des deux processus ne s'en est aperçu.

Les sections critiques

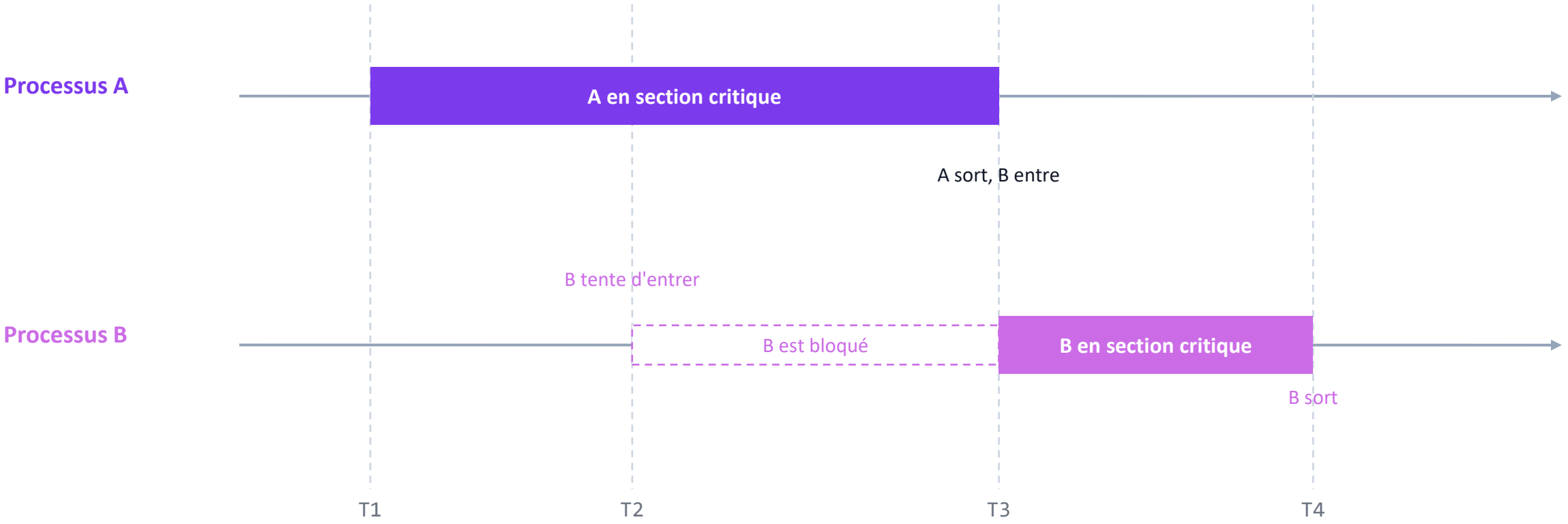
- Une section critique est une portion de code qui accède à une donnée partagée
- Pour éviter les conflits, il faut l'exclusion mutuelle : si un processus est dans sa section critique, les autres en sont exclus
- Attention : A++ n'est pas une seule opération, mais trois (lire, calculer, écrire). Un processus peut être interrompu entre les trois



sections critiques

```
$ objdump -d prog   # ce que devient A++ :
mov  eax, [A]       ; lire
add  eax, 1         ; calculer
mov  [A], eax       ; écrire
```

L'exclusion mutuelle dans le temps



Jamais deux processus en même temps dans leur section critique : c'est tout l'objectif.

Les 4 conditions d'une bonne solution

01

Exclusion mutuelle

Deux processus ne doivent jamais se trouver en même temps dans leur section critique.

02

Aucune hypothèse

On ne suppose rien sur la vitesse ni sur le nombre de processeurs.

03

Pas de blocage extérieur

Un processus hors de sa section critique ne doit pas bloquer les autres.

04

Pas de famine

Aucun processus ne doit attendre indéfiniment pour entrer dans sa section critique.

Comment obtenir l'exclusion mutuelle ?

Avec attente active

Le processus boucle en testant une condition tant qu'il ne peut pas entrer.

1. Masquer les interruptions — réservé au noyau

2. Variable verrou — ne marche pas !

3. Alternance stricte — viole la condition 3

4. Solution de Peterson — correcte, en logiciel

5. Instruction TSL — correcte, avec le matériel

Avec blocage

Le processus est endormi par le noyau et réveillé quand il peut entrer. Pas de CPU gaspillé.

6. Primitives sleep / wakeup — l'idée de base

7. Sémaphores — Dijkstra, 1965

8. Mutex — le verrou d'aujourd'hui

En pratique : on utilise les mutex et les sémaphores fournis par le système (séance 8).

Interruptions et variable verrou

Masquer les interruptions

Sans interruption, plus de changement de processus. Mais on ne peut pas donner ce pouvoir à un programme utilisateur, et cela ne marche pas avec plusieurs processeurs.

verrou.c

```
int verrou = 0;              // partagé

while (verrou == 1)
    ;                        // attendre
verrou = 1;
section_critique();
verrou = 0;
```

1. P1 lit verrou = 0 : il sort de la boucle

2. Interrompu avant verrou = 1 ! P2 lit verrou = 0 et sort aussi

3. P1 et P2 sont tous les deux en section critique

Le verrou ne fait que déplacer le problème : tester puis modifier verrou prend deux opérations, et un processus peut être interrompu entre les deux. C'est la même condition de concurrence que A++.

L'alternance stricte

alternance.c

```c
int tour = 0;      // partagé : à qui le tour ?

/* processus 0 */              /* processus 1 */
while (tour != 0) ;            while (tour != 1) ;
section_critique();            section_critique();
tour = 1;                      tour = 0;
section_non_critique();        section_non_critique();
```

Exclusion garantie

tour vaut 0 ou 1 : un seul processus peut sortir de sa boucle while. Les processus passent chacun leur tour.

Mais la condition 3 est violée.
Si P1 est lent dans sa section non critique, P0, qui a fini, ne peut pas rentrer : il doit attendre que P1 repasse par sa section critique pour remettre tour à 0. Un processus hors de sa section critique bloque l'autre.

La solution de Peterson (1981)

peterson.c

```c
#define N 2
int tour;                   // à qui de céder ?
int interesse[N];           // initialisé à 0

void entrer(int p)          // p = 0 ou 1
{
    int autre = 1 - p;
    interesse[p] = 1;
    tour = p;
    while (tour == p && interesse[autre])
        ;                   // attente active
}

void quitter(int p)
{
    interesse[p] = 0;
}
```

- Chaque processus annonce qu'il est intéressé, puis cède son tour
- Il n'attend que si l'autre est intéressé et que c'est à lui de céder
- Si les deux arrivent en même temps, c'est le dernier à avoir écrit tour qui attend
- Respecte les 4 conditions, sans aide du matériel

Limite actuelle : les processeurs modernes réordonnent les accès mémoire. Sans barrières mémoire, Peterson peut échouer sur une machine multicœur.

L'aide du matériel : TSL

TSL (Test and Set Lock) lit une case mémoire et y écrit 1 en une seule instruction : le processeur garantit qu'aucun autre ne peut s'intercaler.

tsl.asm

```
entrer:
    TSL REGISTRE, VERROU   ; copie VERROU, puis VERROU = 1
    CMP REGISTRE, 0        ; le verrou était-il libre ?
    JNE entrer             ; non : on recommence
    RET                    ; oui : on entre

quitter:
    MOV VERROU, 0          ; libère le verrou
    RET
```

en C11

```
#include <stdatomic.h>
atomic_flag verrou = ATOMIC_FLAG_INIT;

while (atomic_flag_test_and_set(&verrou))
    ;                          // attente active
section_critique();
atomic_flag_clear(&verrou);
```

Les instructions atomiques existent sur tous les processeurs actuels (xchg, lock cmpxchg sur x86). Elles sont la base de tous les verrous.

Éviter l'attente active : sémaphores et mutex

Le défaut de l'attente active

Le processus consomme du CPU pour rien. Pire : s'il est plus prioritaire que celui qui tient le verrou, il peut l'empêcher de le rendre (inversion de priorité).

sleep / wakeup

Idée : un processus qui ne peut pas entrer s'endort, et celui qui sort le réveille. Le noyau gère la file d'attente.

Sémaphore (Dijkstra)

Un compteur avec deux opérations atomiques :
• P (down, sem_wait) : décrémente, ou s'endort si le compteur vaut 0
• V (up, sem_post) : incrémente, et réveille un processus en attente

```
sem_t *s = sem_open("/compte",
          O_CREAT, 0644, 1);
sem_wait(s);
solde = solde - 100;
sem_post(s);
```

Mutex

Un verrou à deux états, libre ou pris. Celui qui le prend doit le rendre. C'est l'outil de base avec les threads.

```
pthread_mutex_t m =
    PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_lock(&m);
solde = solde - 100;
pthread_mutex_unlock(&m);
```

SYNTHÈSE

À retenir

01

pipe

p[0] pour lire, p[1] pour écrire. FIFO, sens unique, capacité finie, lecture effaçante.

02

Fermer

Fermez les extrémités inutiles : sinon pas de fin de fichier, pas de SIGPIPE, et des blocages.

03

dup2

Branche un descripteur sur 0 ou 1 : c'est ainsi que le shell réalise < > et |.

04

mkfifo

Un tube avec un nom : il relie des processus sans lien de parenté.

05

Concurrence

Le résultat dépend de l'ordre d'exécution. Les sections critiques doivent être protégées.

06

Exclusion mutuelle

Peterson et TSL l'obtiennent en attente active ; sémaphores et mutex en endormant.

Ressources pédagogiques

Le manuel

```
$ man 2 pipe
$ man 7 pipe        # règles détaillées
$ man 2 dup2
$ man 3 mkfifo
$ man 7 sem_overview
$ man ipcs
```

man 7 pipe décrit précisément la capacité, PIPE_BUF et le comportement en mode non bloquant.

Lectures

- A. Tanenbaum, Systèmes d'exploitation, Pearson : « Communication interprocessus »
- M. Kerrisk, The Linux Programming Interface : chapitres 44 (tubes) et 53 (sémaphores)
- E. W. Dijkstra, Cooperating Sequential Processes, 1965

lamae

Questions ?

PROCHAINE SÉANCE

CM 04 — Les signaux et les IPC

kill, sigaction, masques de signaux · files de messages, sémaphores, mémoire partagée

Christophe Cruz · christophe.cruz@ube.fr